

NEITZEL — SISTEMA EMPRESARIAL

Auditoria Completa + Estabilização + Agente Supervisor de IA

Este documento define a prioridade de trabalho no sistema **Neitzel — Sistema Empresarial**. O sistema já é grande e possui muitos módulos e funcionalidades. Neste momento, a prioridade NÃO é adicionar novas funcionalidades.

Prioridade: **ENTENDER → ENCONTRAR O QUE ESTÁ QUEBRADO → CORRIGIR → VALIDAR → ORGANIZAR → GARANTIR ESTABILIDADE → DEPOIS EVOLUIR.**

1. Regra mais importante

NÃO ALTERE O SISTEMA DE FORMA AGRESSIVA.

- Não recrie o projeto.
- Não apague funcionalidades.
- Não substitua estruturas existentes sem necessidade.
- Não faça grandes refatorações apenas porque existe uma forma diferente de fazer.
- Não altere algo que já funciona sem entender suas dependências.
- Não adicione novas funcionalidades enquanto houver problemas críticos não resolvidos.

O sistema já é grande. Cada alteração deve considerar o impacto sobre o restante da aplicação.

2. Primeira etapa — Auditoria completa

Antes de implementar qualquer coisa, faça uma auditoria profunda do sistema inteiro. Mapeie:

- Frontend
- Backend
- Banco de dados
- Autenticação
- Usuários
- Permissões
- APIs
- Integrações
- Serviços
- Componentes
- Rotas
- Formulários
- Dashboards
- CRM
- Leads
- Comunicação
- IA
- Automações
- Financeiro
- Clientes

- Agenda
- Produtos
- Estoque
- Acessor
- Configurações
- Notificações
- Outros módulos existentes

Descubra como cada parte depende das outras.

3. Criar um mapa do sistema

Crie uma visão geral da arquitetura. Para cada módulo, identifique:

- O que ele deveria fazer.
- O que realmente está funcionando.
- O que está parcialmente funcionando.
- O que está quebrado.
- O que está apenas visualmente implementado.
- Quais APIs utiliza.
- Quais tabelas utiliza.
- Quais componentes utiliza.
- Quais dependências possui.
- Quais outros módulos dependem dele.

Não assuma que uma tela funcionando visualmente significa que a funcionalidade está funcionando.

4. Testar funcionalidades reais

Procure principalmente por:

- Botão que não executa nada.
- Formulário que não salva.
- Dados que não aparecem.
- Dados que aparecem incorretamente.
- Alterações que não persistem.
- APIs quebradas.
- Erros silenciosos.
- Estados inconsistentes.
- Falhas de autenticação.
- Problemas de permissões.
- Dados duplicados.
- Consultas incorretas.
- Componentes desconectados do backend.
- Funcionalidades parcialmente implementadas.
- Rotas quebradas.
- Integrações configuradas incorretamente.
- Variáveis de ambiente ausentes.

- Erros no console e no servidor.
- Problemas de responsividade.

5. Classificar os problemas

- **CRÍTICO** — Impede o sistema de funcionar ou pode causar perda/corrupção de dados.
- **ALTO** — Uma funcionalidade importante está quebrada.
- **MÉDIO** — A funcionalidade funciona parcialmente ou possui problemas relevantes.
- **BAIXO** — Problemas visuais, pequenos bugs ou melhorias.

Ordem de prioridade: **CRÍTICO** → **ALTO** → **MÉDIO** → **BAIXO**

6. Não confundir interface com funcionalidade

Uma tela bonita não significa que está funcionando. Para cada módulo importante, verifique o fluxo completo:

```
Interface
↓
Ação do usuário
↓
Frontend
↓
API
↓
Backend
↓
Banco
↓
Resposta
↓
Atualização da interface
```

Teste o caminho inteiro.

7. Testar os fluxos principais

CLIENTE: criar cliente → salvar → aparecer na lista → abrir cliente → editar → salvar → confirmar persistência.

LEAD: criar lead → visualizar → alterar status → registrar interação → verificar histórico.

SERVIÇO: criar serviço → agendar → executar → finalizar → registrar valor → verificar financeiro.

PRODUTO: cadastrar produto → entrada no estoque → venda/utilização → baixa no estoque → conferir saldo.

FINANCEIRO: registrar receita → registrar custo → registrar despesa → calcular lucro → conferir dashboard.

8. Banco de dados

Analise cuidadosamente o banco, procurando:

- tabelas incompletas;
- relações incorretas;

- campos duplicados;
- campos sem uso;
- registros inconsistentes;
- problemas de integridade;
- políticas de segurança;
- permissões;
- isolamento entre usuários/empresas;
- consultas ineficientes.

Não faça migrações destrutivas sem necessidade. Antes de alterar estrutura, entenda quem utiliza aquela estrutura.

9. Segurança

Verifique: autenticação, autorização, permissões, isolamento de dados, acesso por usuário, acesso por empresa, APIs protegidas, exposição de dados, secrets, variáveis de ambiente e validação de inputs.

Nenhum usuário deve conseguir acessar dados que não pertencem a ele.

10. Agente Supervisor do sistema

Depois da auditoria e estabilização, criar uma camada de IA chamada **Agente Supervisor**, responsável por acompanhar o funcionamento do sistema enquanto ele estiver online.

Não deve ser apenas um chatbot. Deve funcionar como camada inteligente de **MONITORAMENTO + ASSISTÊNCIA + DIAGNÓSTICO + ORIENTAÇÃO**.

11. Funções do Agente Supervisor

O agente deve conseguir identificar:

- Erros;
- Falhas de API;
- Integrações indisponíveis;
- Processos que falharam;
- Dados inconsistentes;
- Automações que não executaram;
- Problemas recorrentes;
- Comportamentos anormais;
- Problemas de performance;
- Funcionalidades apresentando erros.

12. Monitoramento inteligente

Não basta registrar milhares de logs: o agente deve interpretar sinais.

■■ Detectei um problema na integração X. Foram identificadas 8 falhas nos últimos 10 minutos. A integração parece estar indisponível.

■■ A automação de atualização de leads apresentou falhas. Última execução: 14:32. Motivo: erro de comunicação com o serviço X.

13. Agente como assistente do usuário

- "Você tem 7 leads sem acompanhamento."
- "Existem 3 tarefas atrasadas."
- "O estoque do Produto X está abaixo do mínimo."
- "Sua agenda amanhã está com 6 atendimentos."
- "Você teve queda de conversão esta semana."
- "Existem oportunidades paradas há mais de 5 dias."

14. Dicas inteligentes

- Sugestão: você possui 12 leads que não receberam contato há mais de 48 horas. Recomendo priorizar esses leads hoje.
- Atenção: o Produto X está vendendo rapidamente e o estoque está próximo do mínimo.

As sugestões devem ser úteis. Não bombardear o usuário com notificações.

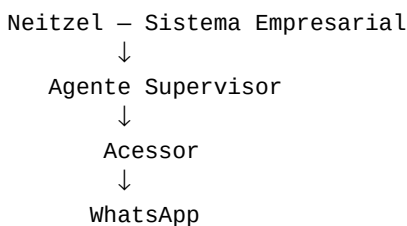
15. Níveis de autonomia

- **OBSERVAR** — pode monitorar e analisar.
- **SUGERIR** — pode recomendar ações.
- **EXECUTAR** — pode executar ações previamente autorizadas.
- **CRÍTICO** — sempre pedir aprovação humana.

O agente nunca deve tomar decisões financeiras, apagar dados ou realizar ações irreversíveis sem autorização explícita.

16. Agente + Acessor

O Acessor é a interface operacional; o Agente Supervisor é a inteligência de monitoramento.



O usuário pode receber informações pelo sistema e, futuramente, pelo WhatsApp.

17. Centro de Inteligência

Área dentro do sistema para o agente, exibindo:

- **Saúde do sistema:** ■ Sistema operacional
- **Alertas:** 2 problemas detectados
- **Sugestões:** 3 recomendações
- **Atividades:** 18 processos monitorados
- **Integrações:** WhatsApp conectado, IA conectada, Banco operacional

Os problemas devem ser exibidos com: gravidade, módulo, horário, descrição, possível causa, status e ação recomendada.

18. Histórico do agente

Registrar: problema detectado, quando ocorreu, módulo afetado, diagnóstico, ação sugerida, ação executada e resultado. Isso permite entender o comportamento do sistema ao longo do tempo.

19. Performance

Monitorar tempo de resposta, APIs lentas, consultas lentas, erros, falhas recorrentes, consumo excessivo e processos demorados.

Não otimizar prematuramente: primeiro medir, depois corrigir os pontos realmente problemáticos.

20. Princípio de segurança do agente

O agente nunca deve: inventar dados; fingir que corrigiu algo; apagar dados automaticamente; alterar banco sem controle; alterar permissões; expor informações privadas; executar ações críticas sem aprovação.

Se não tiver certeza, **deve informar que não tem certeza**.

21. Sistema de saúde

Banco de dados	■
Autenticação	■
API principal	■
WhatsApp	■
IA	■
Automações	■
Estoque	■
Financeiro	■

O objetivo é identificar rapidamente onde existe um problema.

22. Regra para novas funcionalidades

- 1 Entender a necessidade.
- 2 Verificar se já existe algo semelhante.
- 3 Verificar dependências.
- 4 Planejar banco/API/frontend.
- 5 Implementar.
- 6 Testar.
- 7 Verificar impacto em módulos existentes.
- 8 Validar funcionamento.
- 9 Somente então considerar concluída.

23. Regra contra "telas fake"

É proibido considerar uma funcionalidade concluída apenas porque a tela existe, o botão existe, aparece uma mensagem de sucesso ou os dados aparecem temporariamente.

Uma funcionalidade só está pronta quando **Frontend + Backend + Banco + Regras + Persistência + Tratamento de erros** estiverem funcionando corretamente.

24. Relatório final da auditoria

- **Estado geral:** percentual aproximado de módulos funcionando, parcialmente funcionando, com problemas e não implementados.
- **Problemas críticos:** listar primeiro.
- **Problemas importantes:** listar depois.
- **Melhorias:** separar melhorias de bugs.
- **Riscos:** pontos que podem quebrar outras partes do sistema.
- **Plano de correção:** definir ordem de correção.

25. Não implementar tudo de uma vez

Trabalhe por etapas:

AUDITORIA

- CORREÇÃO DOS CRÍTICOS
- CORREÇÃO DOS IMPORTANTES
- VALIDAÇÃO
- AGENTE SUPERVISOR
- NOVAS FUNCIONALIDADES

26. Regra final

Aja como um **engenheiro responsável pela estabilidade do sistema**, não como alguém que apenas recebe comandos e escreve código.

- Antes de modificar: **PENSE.**
- Antes de criar: **VERIFIQUE SE JÁ EXISTE.**
- Antes de apagar: **ANALISE DEPENDÊNCIAS.**
- Antes de alterar: **VERIFIQUE IMPACTOS.**
- Depois de implementar: **TESTE.**
- Depois de testar: **VERIFIQUE SE OUTRAS PARTES FORAM AFETADAS.**

O objetivo é um sistema **ESTÁVEL + FUNCIONAL + INTELIGENTE + SEGURO + ORGANIZADO + ESCALÁVEL**. O Neitzel — Sistema Empresarial deve ser tratado como um produto real em produção, priorizando a estabilidade acima da velocidade de implementação.